

不同开发团队自身的经验，建立合适的经验模型。但毋庸置疑，软件良好的设计方法无疑对软件的可维护性有着积极的作用。比如模块化设计中模块划分的合理性，如模块的可分解性（Decomposability）、可组合性（Composability）、可理解性（Understandability）、可持续性（Continuity）和保护性（Protection）等，以及模块的高内聚和低耦合性等，在设计方面应审慎考量，并加以应用，以便为软件系统的后期维护提供帮助。

3. 软件维护的分类

在系统运行过程中，软件需要维护的原因是多样的。根据维护的原因不同，可以将软件维护分为以下4类：

（1）改正性维护。为了识别和纠正软件错误、改正软件性能上的缺陷、排除实施中的误使用，应当进行的诊断和改正错误的过程称为改正性维护。

（2）适应性维护。在使用过程中，外部环境（新的硬、软件配置）、数据环境（数据库、数据格式、数据输入/输出方式、数据存储介质）可能发生变化。为使软件适应这种变化，而去修改软件的过程称为适应性维护。

（3）完善性维护。在软件的使用过程中，用户往往会对软件提出新的功能与性能要求，为了满足这些要求，需要修改或再开发软件，以扩充软件功能、增强软件性能、改进加工效率、提高软件的可维护性。这种情况下进行的维护活动称为完善性维护。

（4）预防性维护。预防性维护是指预先提高软件的可维护性、可靠性等，为以后进一步改进软件打下良好基础。通常，预防性维护可定义为“把今天的方法学用于昨天的系统以满足明天的需要”。也就是说，采用先进的软件工程方法对需要维护的软件或软件中的某一部分（重新）进行设计、编码和测试。

15.4.2 软件维护的影响因素

软件维护阶段占整个软件生命周期60%以上的时间，因此，分析影响软件维护的因素，提高软件可维护性就显得十分重要。软件维护的影响因素很多，主要有以下几个方面：

（1）业务因素。因为系统在线运行，某些系统要保证7×24h运行，维护人员必须寻找一种途径，在不影响用户业务的情况下实现改动。

（2）理解的局限性。统计数据显示，50%左右的软件维护工作量花在理解要修改的软件上。有时用户的理解也会出现问题，约有超过半数的维护问题源自用户技能或理解的缺乏。同时，维护人员也要具备一些人际技巧，努力理解不同用户的思维方式，以说服的方式处理掉一些问题。

（3）对待维护的优先级问题。有时开发商会倾向于维持现有系统的运行，而客户更迫切地需要新功能，甚至一个新系统。

（4）维护人员的积极性。通常，维护人员被认为是第二阶层，程序员大都认为设计和开发比维护工作更具技巧性和挑战性。

（5）测试的困难。测试人员很难预测设计或代码改动带来的影响，使得测试很难做到充分。另外，有些系统只能在测试环境或备份系统中进行测试，上线后则不允许测试，由于无法精确地再现真实环境，也使得测试有一定的局限性。